Descripción del código 10 pokemon más difíciles de capturar

El código no solo grafica datos, sino que actúa como un transformador de variables para generar un índice de riesgo operativo. Estas son sus funciones principales:

1. Sanitización de Tipos y Resiliencia de Datos

El primer paso crítico es la conversión forzada de columnas mediante pd.to_numeric.

- Lógica de Fallo Controlado: El uso de errors='coerce' garantiza que cualquier anomalía en el CSV (espacios, texto o datos corruptos) no detenga la ejecución, convirtiéndolos en valores nulos que el algoritmo puede ignorar de forma segura.

- Objetivo: Asegurar que las operaciones aritméticas posteriores se realicen sobre una matriz numérica pura.

2. Inyección de Contexto Externo (Penalizadores)

Dado que el dataset es una captura estática de estadísticas, el código utiliza un mapeo de comportamiento (mecanicas_reales).

- Función: Asigna pesos específicos a unidades que presentan riesgos no cuantificables en el CSV, como el suicidio por retroceso, la huida instantánea o la regeneración de salud.

- Propósito: Corregir el sesgo de los datos planos para que Pokémon como Abra o Beldum escalen en dificultad real.

3. Motor Aritmético de Complejidad (calcular_complejidad)

Es el núcleo del script. Procesa cada fila del dataset mediante una fórmula multivariable:

- Inversión de Probabilidad: Convierte el ratio de captura en un valor de dificultad (a menor ratio, mayor índice).

- Escalamiento de Persistencia (HP/10): Trata los puntos de salud como un "presupuesto de turnos". Un valor alto incrementa la duración del combate, elevando el riesgo de fallo sistémico.

- Factor de Potencia (BST/100): Utiliza el total base como una variable de correlación para medir la resistencia intrínseca de la especie.

4. Semántica Visual y Degradado de Alerta

El renderizado no es solo estético; utiliza la intensidad del color para comunicar niveles de riesgo.

- Mecánica: Genera un arreglo de colores Reds proporcional al rango de los datos.

- Lógica de Usuario: Mapea automáticamente los índices más altos con los rojos más oscuros. Esto permite una lectura rápida de alerta: el ojo identifica el peligro por la saturación del color antes de procesar el número exacto.

5. Optimización de Interfaz y Persistencia

El código limpia la visualización eliminando los bordes innecesarios (ax.spines) y aplicando un fondo de alto contraste (#0B0E11).

- Exportación: Configura una densidad de pixeles de 300 DPI, asegurando que la gráfica mantenga su nitidez técnica al ser integrada en documentos de investigación o presentaciones profesionales de la UPSRJ.